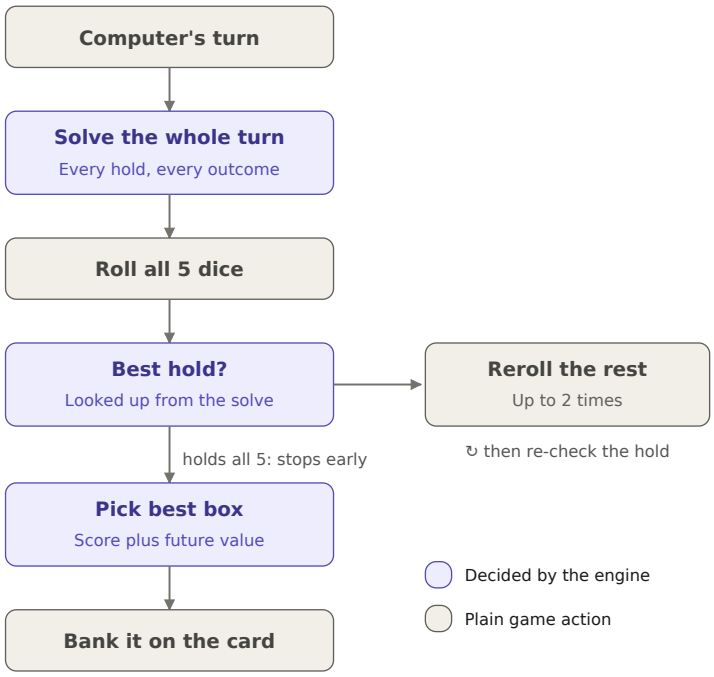
How the Yahtzee computer player works

Based on **src/engine.js** and **runAiTurn** in **src/app.js** (slayyyyer/yahtzee)

The computer is not a rule-of-thumb bot or a neural network. It is an exact probability solver. Once per turn it solves the whole turn with backward induction, and every later decision (which dice to hold, whether to stop, which box to score) is a lookup into that solution.

1. The computer's turn



Purple boxes are decided by the engine; gray boxes are plain game actions.

2. How the solver values a turn



The solver starts at the end of the turn and works backward. Scoring a final roll is worth points plus the value of the boxes still open. The last reroll is worth the best hold, averaged over every possible outcome of the dice you pick up. The first reroll repeats the same step one level earlier.

Why it works

- There are only 252 distinct five-dice results (order does not matter), so every hold and every outcome can be checked exactly in about 2 ms.
- If holding all 5 dice is the best option, the computer stops instead of rerolling for the sake of it.
- The value of the rest of the game comes from calibrated per-box values plus a fitted model of the 35-point upper-bonus odds.
- The coach shown to the human player uses the same engine.

The math behind it

A Bellman equation solved by backward induction (expectimax) within one turn, with an approximate value function for the rest of the game.

1. Dice probabilities

A roll is a multiset of faces with counts $c_1 \ldots c_6$ summing to n. Its probability is multinomial. For all five dice this gives 252 states s, each with probability P(s). Holding k and rerolling the other n dice gives outcome e with probability P(e); the new state is k + e.

$$P(c) = \frac{n!}{c_1!\, c_2! \cdots c_6!} \cdot \frac{1}{6^n}$$

2. Final roll (zero rerolls left)

With open boxes B and upper total u. Here u′ is u plus the score if c is an upper box, otherwise u. Y(s) is 1 if s is five of a kind and the Yahtzee box already holds 50, otherwise 0.

$$V_0(s) = \max_{c \in B} [\, \text{score}_c(s) + F(B \setminus \{c\},\, u') + 100 \cdot Y(s) \,]$$

3. Each reroll (backward induction)

The best hold is the argmax; ties go to the larger hold, which is why the computer stops when keeping all 5 dice is optimal.

$$V_r(s) = \max_{k \subseteq s} \sum_e P(e)\, V_{r-1}(k+e), \qquad r = 1, 2$$

Expected value of a whole turn:

$$E = \sum_s P(s)\, V_2(s)$$

4. Value of the rest of the game

This is the approximation. The w_c are 13 calibrated per-box values, seeded with each box's exact solo expectation and tuned by self-play policy iteration. p(B, u) is the upper-bonus probability: 1 if u ≥ 63, 0 if no upper boxes are open, otherwise a logistic fit.

$$F(B, u) = \sum_{c \in B} w_c \;+\; 35 \cdot p(B, u)$$

$$p = \frac{1}{1 + e^{-z}}, \qquad z = \beta_0 + \sum_i \beta_i \frac{x_i - \mu_i}{\sigma_i}$$

The features x_i are the points still needed (63 − u), the sum of solo EVs of the open upper boxes, the number of open upper boxes, the total open boxes, three interaction terms (need², soloSum², need·soloSum) and the average solo EV. The μ, σ and β constants are the fitted values in BONUS_MODEL.

5. Final score

$$\text{total} = \text{upper} + \text{lower} + 35 \cdot [\text{upper} \geq 63] + 100 \cdot (\text{extra Yahtzees})$$

Would perfect play be possible?

Yes, for solitaire expected score. The single-player game is a solved problem.

What the engine is missing

The solve inside a turn is already exact. The only approximation is F(B, u), the value of the boxes left after the turn. Perfect play replaces it with the exact value W of every possible game state:

$$W(B, u, y) = \sum_{s} P(s)\, V_2^*(s; B, u, y)$$

$$V_0^*(s) = \max_{c \in B}[\, \text{score}_c\,(s) + 100\, Y(s) + W(B \setminus \{c\},\, u',\, y')\,]$$

Here B is the set of open boxes, u the upper total (0 to 63, anything above is capped), and y a flag for whether the Yahtzee box holds 50, which is needed to know if the +100 bonus is live. W is computed by dynamic programming, starting from the empty card (W = 0) and working up to the full card. The per-turn solver you already have is reused for each state.

How big is it?

Item	Size
Open-box subsets	2^13 = 8,192
Upper totals	64 (0 to 63, capped)
Yahtzee-scored-50 flag	2
Upper bound on states	about 1.05 million (fewer are reachable)
Table size	about 4 to 8 MB as float32 / float64
Compute (rough estimate)	tens of minutes single-threaded in Node, since one turn solve is ~2 ms

What you would gain

The README reports about 250.39 points for the current engine against 254.59 for perfect play, so exact play is worth roughly 4 points per game (under 2%).

Caveats

- The table must match this repo's rules. The code does not enforce the official joker restriction (an extra Yahtzee must use its matching upper box when open), so the table would be built for that variant, not the standard one.
- Perfect expected score is not the same as perfect winning. This is a head-to-head game, so the strategy that maximizes win probability differs from the one that maximizes points (for example when trailing late). Two-player Yahtzee is not solved exactly.
- The table has to ship with the game. The README promises no fetch and no build step, so it would need to be inlined (a few MB as base64) or loaded as a script file.